Общие данные: гонки и мьютексы

Роман Кутимский



Тест по прошлой лекции



Отсканируйте QR-код

или наберите ссылку в браузере:

`forms.gle/CjuZYoLKbXgnFn2u5`

ЧАСТЬ 2

Планировщик

Кто решает, какой поток когда работает?



Ускорение растёт до числа ядер, дальше — плато



≈ 4,0×

4 потока на 4 ядрах — почти идеал

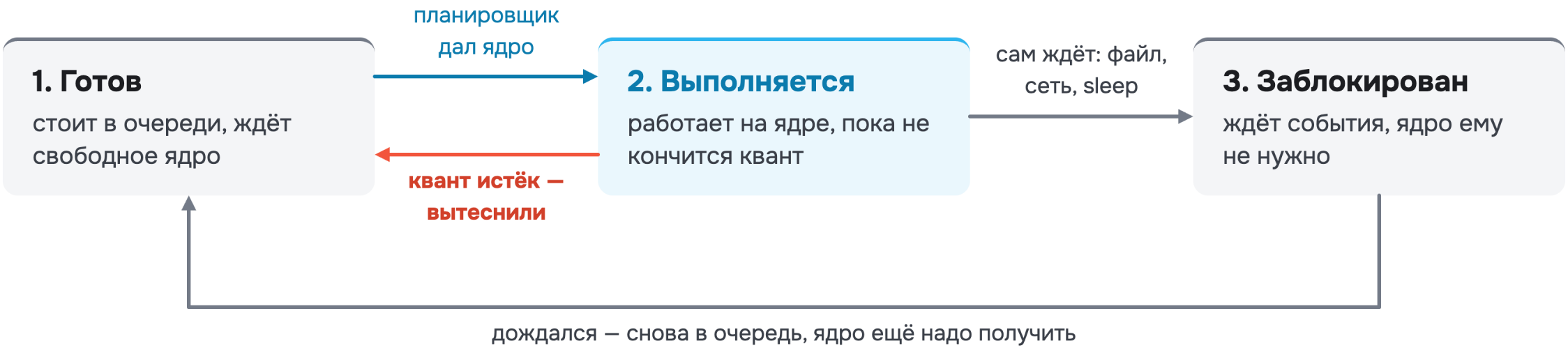
3,35×

5 потоков: в конце одно ядро досчитывает, три ждут

≈ 3,9×

6–16 потоков: плато. Больше 4× на 4 ядрах не бывает

Жизнь потока: три состояния

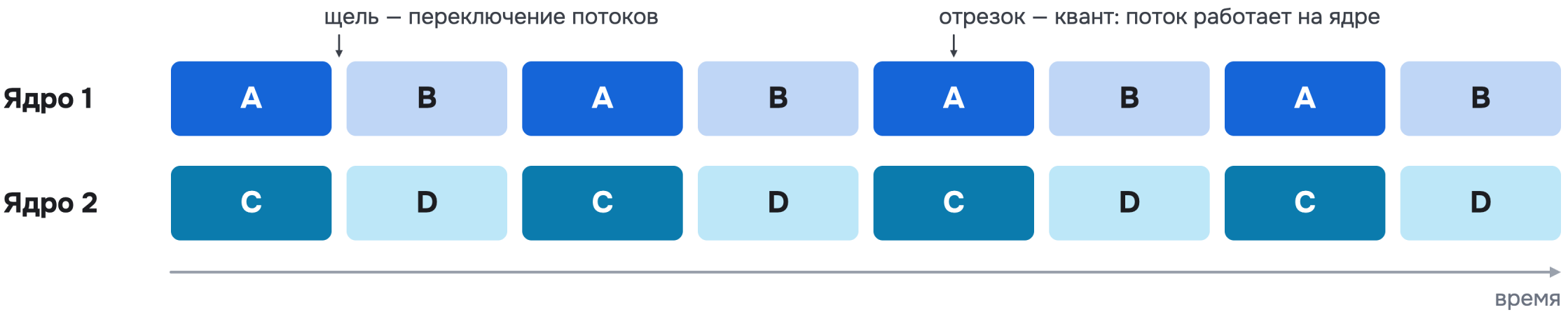


Планировщик — часть операционной системы, которая раздаёт ядра потокам.

Квант — кусочек времени на ядре: единицы–десятки миллисекунд.

Вытеснение (красная стрелка) — ОС забирает ядро, не спрашивая поток, в любой момент.

4 потока на 2 ядрах: планировщик переключает



Работает урывками

каждый поток получает ядро на квант, потом ждёт

Быстрее не станет

ядер всё равно 2 — это и есть плато лабы 2

Переключения — время

каждое переключение стоит микросекунды

Замер на машине с 15 ядрами: 15 потоков — 192 мс, 960 потоков — 206 мс. Работа та же, лишние потоки только ждут.

ЧАСТЬ 3

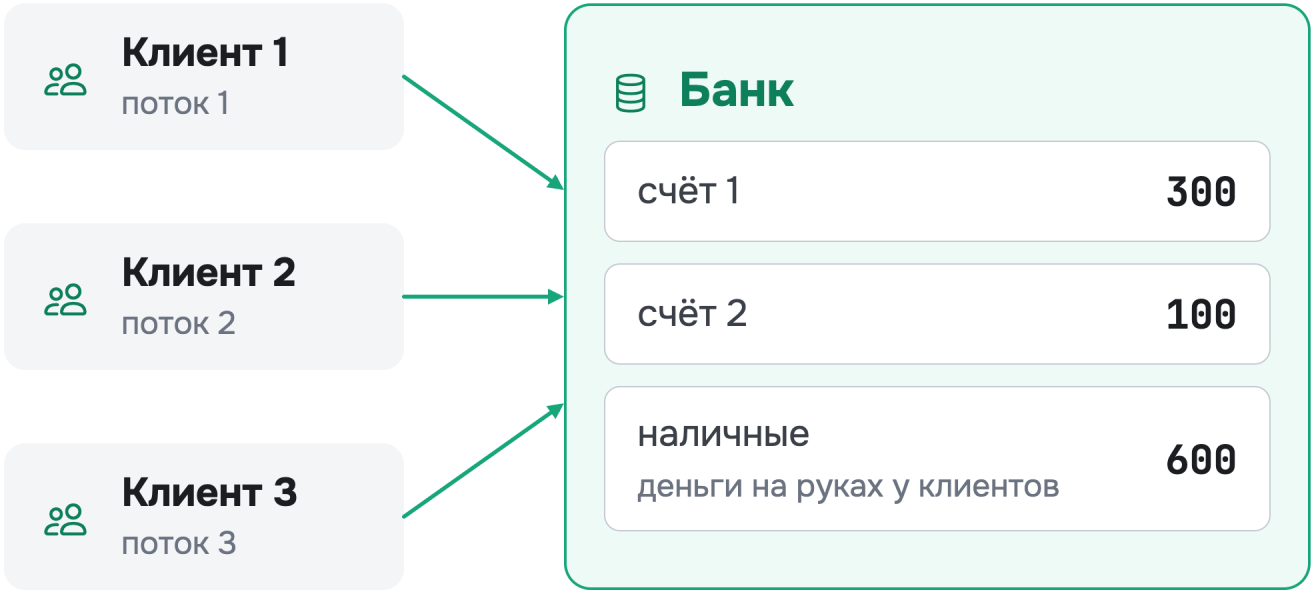
Банк

Что мы будем строить в лабе 3?

- Счета и клиенты
- Операции: положить, снять, узнать баланс
- Правила банка



Банк: счета, клиенты и операции



Операции

`Deposit(счёт, сумма)`
положить наличные на счёт

`Withdraw(счёт, сумма)`
снять со счёта в наличные

`GetBalance(счёт)`
узнать, сколько на счёте

Все клиенты работают с одним банком — одновременно.

Два правила банка

1. Деньги не появляются и не пропадают

наличные + все счета — всегда одно и то же

наличные + счёт 1 + счёт 2

```
600 + 300 + 100 = 1000
```

после Deposit(1, 100):

```
500 + 400 + 100 = 1000
```

2. Баланс не уходит в минус

нельзя снять больше, чем есть на счёте

на счёте 2 лежит 100

```
Withdraw(2, 80) → 20
```

можно

```
Withdraw(2, 150) → -50
```

нельзя

Такие правила называются **инварианты** — то, что должно быть верно всегда, между любыми операциями.

ЧАСТЬ 4

Гонки

Как два клиента ломают банк, ничего плохого не делая?

- Что потоки делят между собой
- Потерянное обновление
- «Проверил, потом сделал»
- Почему гонки трудно поймать



Что потоки делят между собой

Процесс — одна запущенная программа

Глобальные переменные
объявлены вне функций, живут всю программу

Куча
память из new, а также элементы vector и string

ОБЩИЕ — видны всем потокам

Стек
потока 1

```
int i;
double x;
```

Стек
потока 2

```
int i;
double x;
```

Стек
потока 3

```
int i;
double x;
```

ЛОКАЛЬНЫЕ — у каждого потока свои

⚠ **Но:** если передать в поток ссылку или указатель ([&], std::ref), локальная переменная тоже становится общей.

Два клиента кладут по 100 на один счёт

```
int balance = 0;          // на счёте 0
// два клиента — два потока
std::jthread a([&] { balance += 100; });
std::jthread b([&] { balance += 100; });
a.join();
b.join();
std::cout << balance;     // сколько?
```

[&] — оба потока берут balance по ссылке: он общий

ждали: 100 + 100

200

иногда получаем

100

Ошибка: одно пополнение пропало

Куда оно делось? Смотрим на balance += 100 внимательнее.

balance += 100 — это три шага

C++

```
balance += 100;
```

выглядит как одно действие

ПРОЦЕССОР

1	**Прочитать**	взять balance из памяти в регистр: 0

здесь поток могут остановить

2	**Прибавить**	в регистре: 0 + 100 = 100

и здесь тоже

3	**Записать**	из регистра обратно в balance: 100

Регистр — маленькая ячейка внутри ядра, где идёт счёт. Процессор берёт число из памяти в регистр, считает там и кладёт обратно.

Поток могут остановить между любыми двумя инструкциями. Вклинится другой поток — пополнение пропадёт.

Как теряется пополнение

Шаг	Поток A	Поток B	balance в памяти
1	читает balance → 0		0
2		читает balance → 0	0
3	считает 0 + 100 = 100		0
4		считает 0 + 100 = 100	0
5	записывает 100		100
6		записывает 100	100 — пополнение A затёрто

Два пополнения по 100 — а на счёте 100

Это называется **потерянное обновление** (lost update): одна запись затёрла другую.

Когда два обращения конфликтуют

КОНФЛИКТ — ВСЕ ТРИ СРАЗУ:

1. Одна и та же переменная (ячейка памяти)
2. Хотя бы одно обращение — запись
3. Обращения из разных потоков

ОПАСНО, ЕСЛИ ЕЩЁ И:

4. Ничто не задаёт порядок между ними

Пример	Опасно?	Почему
Два потока делают balance += 100	да	оба пишут, порядка нет
Один пишет balance, другой читает GetBalance()	да	чтение тоже считается!
Два потока только читают rate	нет	никто не пишет
Поток i пишет только results[i]	нет	у каждого своя ячейка
main пишет x = 5, потом создаёт поток; поток читает x	нет	конфликт есть, но порядок задан: создание потока

Обращение — чтение или запись переменной. Конфликт (1–3) + нет порядка (4) = **гонка данных**.

Две разные ошибки

RACE CONDITION

Состояние гонки

Ошибка логики: результат зависит от того, какой поток успел первым.

Бывает, даже когда каждая операция сама по себе защищена и гонки данных нет.

Пример: -60 после двух снятий по 80

DATA RACE

Гонка данных

Конфликт из прошлого слайда, и между обращениями нет порядка.

В C++ это **неопределённое поведение (UB)**: стандарт не обещает вообще ничего. Может работать, может выдать мусор, может упасть.

Пример: balance += 100 в двух потоках

Неопределённое поведение (undefined behavior, UB) — компилятор считает, что такого в программе не бывает, и вправе сделать что угодно.

Обе — ошибки.
Чинить нужно обе.

Проверил, потом сделал: баланс −60

```
if (GetBalance() ⩾ 80)    // проверил
    Withdraw(80);         // сделал
```

На счёте 100

Два клиента одновременно снимают по 80

Шаг	Поток A (клиент 1)	Поток B (клиент 2)	Баланс
1	проверяет: 100 ≥ 80 — да		100
2		проверяет: 100 ≥ 80 — да	100
3	снимает 80		20
4		снимает 80	-60

⚠ **−60. Правило «не в минус» нарушено**

Почему гонки трудно поймать

Редко

Окно между шагами — несколько наносекунд. Потоки попадают в него редко.

Каждый запуск по-разному

Всё зависит от того, как планировщик раздал ядра. Точно повторить ошибку нельзя.

«У меня работает»

На ноутбуке с 4 ядрами не видно, на сервере с 64 ядрами — падает.

Прячется от отладки

Отладчик и вывод на экран (cout) меняют тайминги — и ошибка исчезает.

Гонку нельзя «потестировать до отсутствия». Её надо исключить правильным кодом.

ЧАСТЬ 5

Мьютекс

Как пустить к счёту только один поток за раз?

- Комната с одним ключом
- Как потоки ждут друг друга
- Чиним потерянное обновление
- Как второй поток видит запись первого



Комната с одним ключом

```
std::mutex m;      // мьютекс = ключ
int balance = 0;   // общие данные
// каждый поток делает:
m.lock();          // взять ключ
balance += 100;    // критическая секция
m.unlock();        // отдать ключ
```



мьютекс — от mutual exclusion, «взаимное исключение»

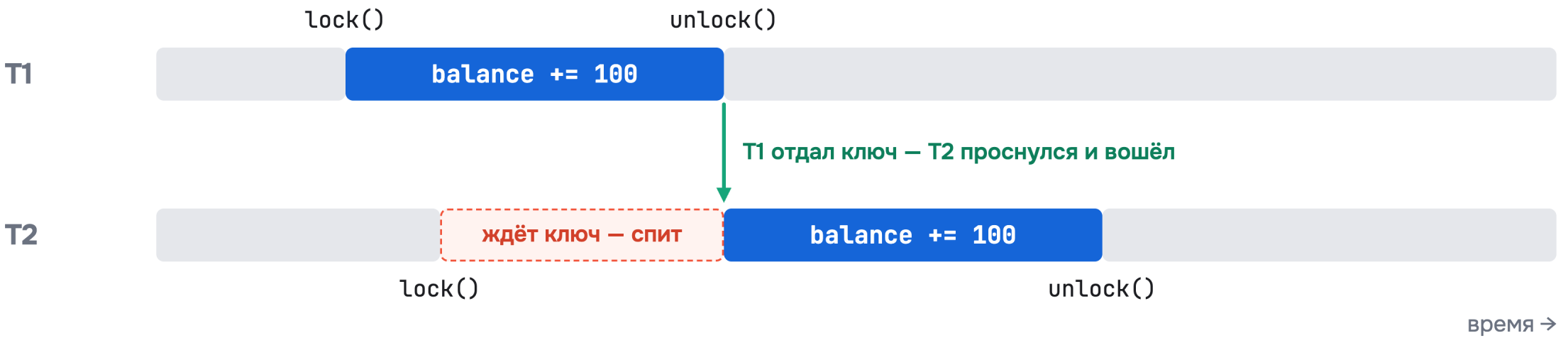
Что такое критическая секция

Кусок кода, который трогает **общие данные**. Выполнять его может **только один поток за раз** — тот, у кого ключ.

Зачем: без ключа — ошибка

Шаги «прочитать – прибавить – записать» двух потоков перемешаются — и на счёте 100 вместо 200.

Как потоки ждут друг друга



`m.lock()`
взять ключ. Занят — поток ждёт, пока его не отдадут

`m.unlock()`
отдать ключ и разбудить одного из ждущих

Ждущий поток
в состоянии «заблокирован»: спит и не тратит ядро

Чиним потерянное обновление

```cpp
#include <mutex>

int balance = 0;
std::mutex m;   // охраняет balance

// в каждом из двух потоков:
m.lock();         // взять ключ
balance += 100;   // внутри — один поток
m.unlock();       // отдать ключ
```

было, без мьютекса — ошибка

иногда 100

стало, с мьютексом — верно

всегда 200

Мьютекс один на оба потока — оба берут один и тот же ключ **m**.

Три шага «прочитать – прибавить – записать» теперь не перемешиваются: второй поток ждёт, пока первый сделает все три.

А как T2 видит то, что записал T1?

ЯДРО 1 · ПОТОК T1

```
balance = 100;
```

записал и отдал ключ

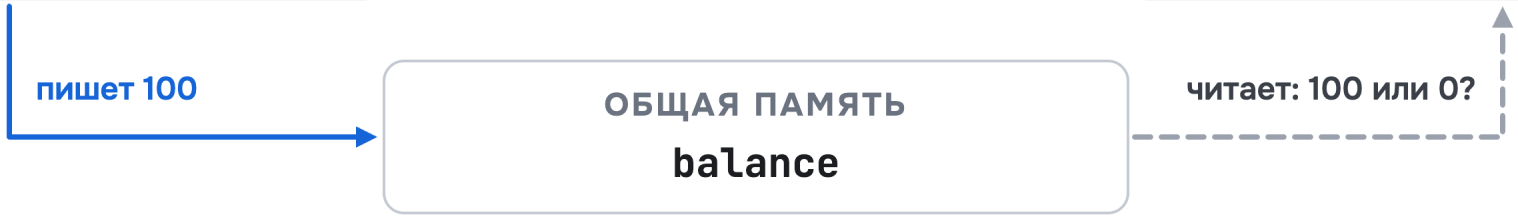
Откуда гарантия, что T2 прочитает 100, а не старый 0?

у ядер свои кэши и буферы, а компилятор переставляет команды

ЯДРО 2 · ПОТОК T2

```
cout << balance;
```

взял ключ и читает



Мьютекс делает две вещи:

✓ **1. Не пускает второй поток**

Это мы уже видели: шаги двух потоков не перемешиваются.

🔍 **2. Следующий видит записи**

Кто взял ключ, видит всё, что записал прошлый владелец.

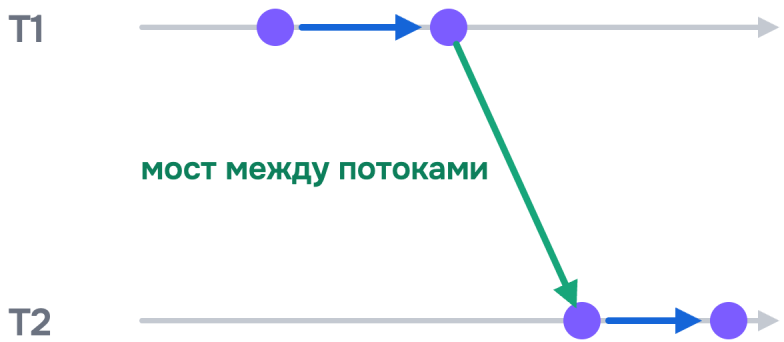
ЧАСТЬ 6

Happens-before

«происходит раньше»

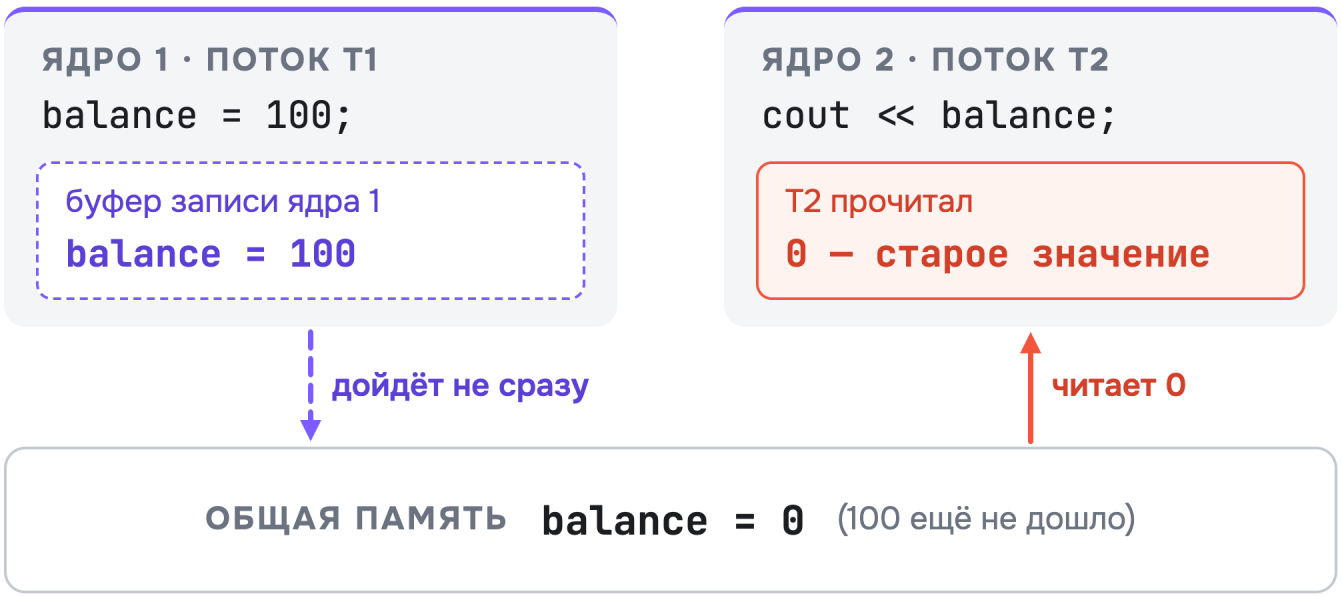
Как доказать, что в программе нет гонки?

- Почему между потоками нет «раньше» само по себе
- Стрелки внутри потока и между потоками
- Цепочки стрелок
- Применяем к лабам и мьютексу



Между потоками нет «раньше» само по себе

1. Запись доходит не сразу



2. Команды переставляются

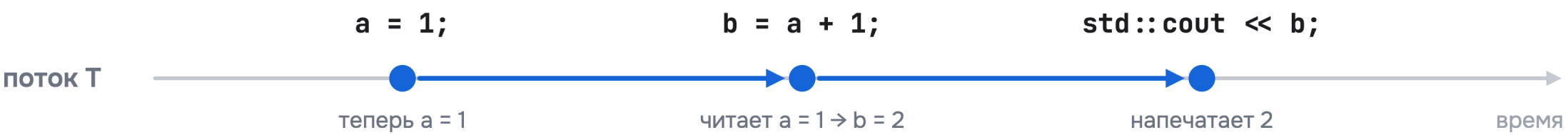
Компилятор и процессор меняют местами независимые команды — так быстрее.

в коде		на деле
`a = 1;` `b = 2;`	→	`b = 2;` `a = 1;`

Сам поток разницы не заметит.
Другой поток — может.

Порядок строк в одном потоке ≠ порядок, в котором их видит другой поток

Внутри потока: порядок строк



sequenced-before — «идёт раньше в коде»

Внутри одного потока каждая строка видит результат всех строк выше. На схемах — **синяя стрелка**.

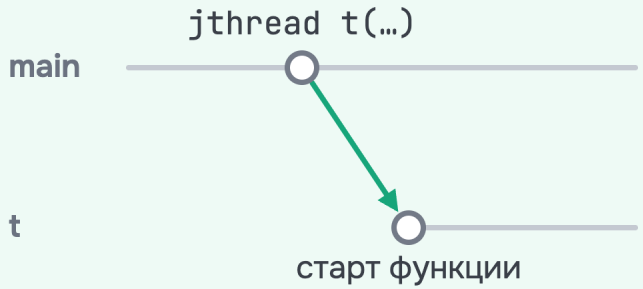
Оговорка

Компилятор и процессор могут переставлять команды ради скорости — но только так, что сам поток этого не заметит.

Вывод

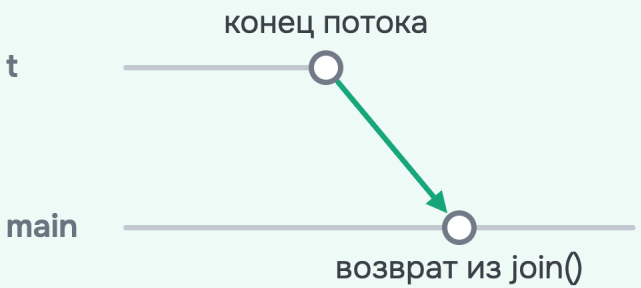
Внутри одного потока всё как в обычной программе. Трудности начинаются, когда потоков два.

Мосты между потоками



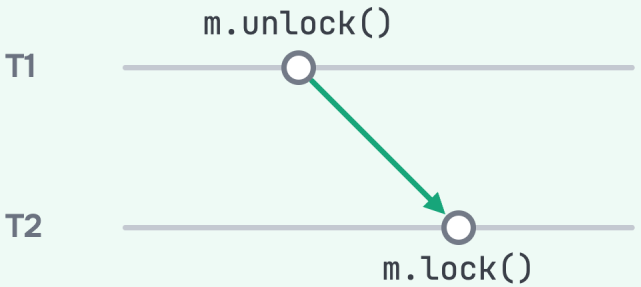
Создание потока

Всё, что main сделал до создания потока, видно внутри его функции.



Конец потока → join()

Всё, что сделал поток, видно в main после возврата из join().



unlock() → lock()

Всё, что поток сделал до unlock(), видит тот, кто следующим возьмёт **тот же** мьютекс.

synchronizes-with — «синхронизируется с»: зелёная стрелка-мост между потоками.

Что было до начала стрелки, видно после её конца.

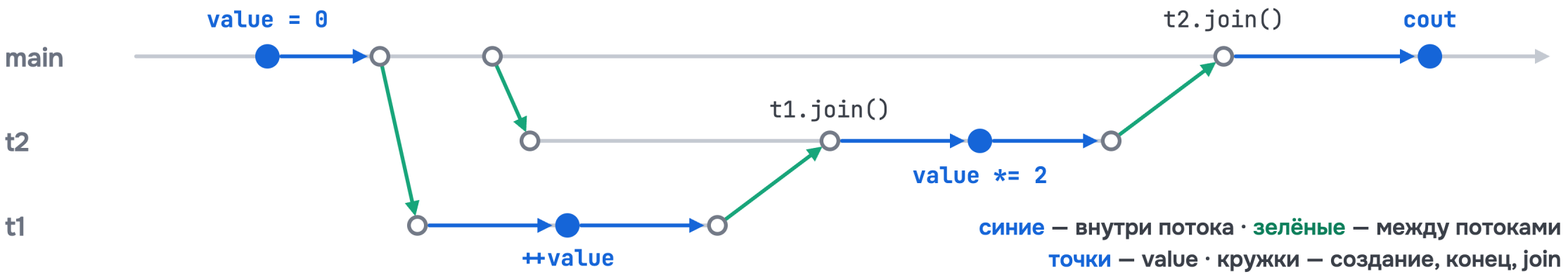
Happens-before — это цепочка стрелок

```
int value = 0;
std::jthread t1([&] { ++value; });
std::jthread t2([&] { t1.join(); value *= 2; });
t2.join();
std::cout << value;
```

A happens-before B

«A происходит раньше B»: от A до B можно дойти по стрелкам — синим и зелёным. Тогда B видит A и всё, что было до A.

Цепочка непрерывна → всегда напечатает 2.



Точное правило гонки данных

Конфликт
A и B — одна ячейка, есть запись, разные потоки

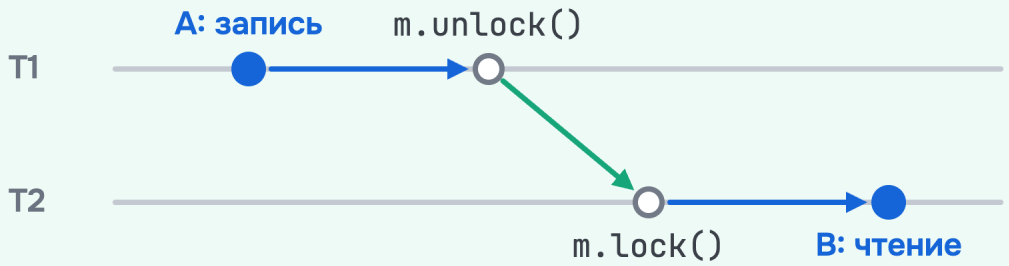
+

Нет цепочки стрелок
ни от A к B, ни от B к A

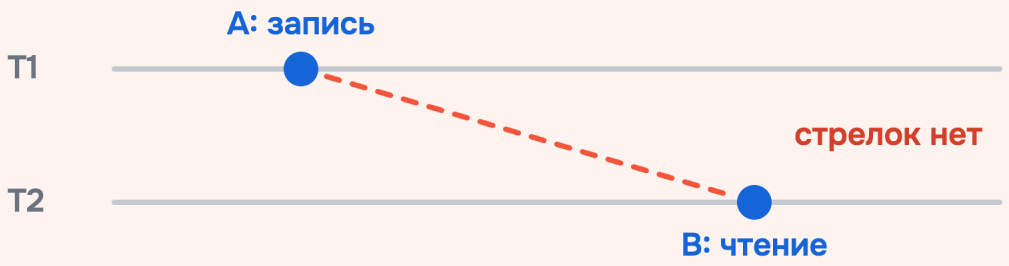
=

Data race = UB
гонка данных: неопределённое поведение

Гонки нет: путь от A к B есть



Гонка: пути нет ни туда, ни обратно

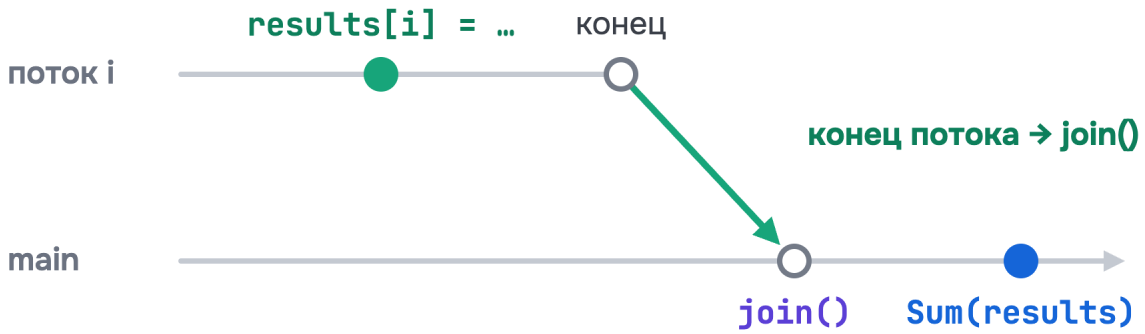


Точнее: «ничто не задаёт порядок» значит «нет цепочки стрелок».

Проверить код на гонку: нарисуйте бусы и поищите путь от A до B.

Почему в лабах 1–2 всё было правильно

```
std::vector<double> results(n);
for (int i = 0; i < n; ++i)
    threads.emplace_back([&, i] {
        results[i] = Work(i);  // своя ячейка
    });
for (auto& t : threads) t.join();
double sum = Sum(results);  // после join
```



1. Каждый поток пишет в свою ячейку

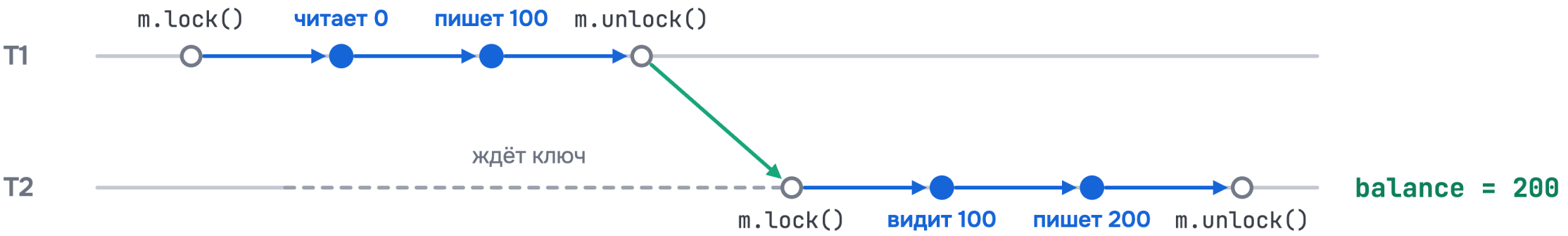
Поток 0 — только в results[0], поток 1 — только в results[1]. Общих ячеек нет — значит, нет и конфликта.

2. main читает только после join()

Конец потока → join() — это стрелка. Поэтому main видит всё, что поток записал.

Мьютекс не нужен: потоки не делят ячейки, а порядок даёт join()

Почему мьютекс чинит баланс



Кто первый — решает мьютекс

Может первым войти T2 — тогда стрелка пойдёт от его unlock() к lock() в T1. Но стрелка будет всегда.

Цепочка есть → гонки нет

Любые два обращения к balance связаны стрелками. T2 видит запись T1: читает 100, а не 0.

Запись T1 видна T2 благодаря стрелке unlock() → lock().

ЧАСТЬ 7

Мьютекс на практике

Как правильно написать потокобезопасный банк?

- std::mutex и lock_guard
- Что именно защищать
- Потокобезопасный интерфейс
- shared_mutex и atomic



std::mutex: три метода

lock()

Взять ключ. Занят — ждать: поток засыпает, пока ключ не отдадут.

unlock()

Отдать ключ. Один из ждущих потоков просыпается и забирает его.

try_lock()

Попробовать: свободно — взял, вернёт true; занято — сразу false.

ПРИМЕР С TRY_LOCK

```
#include <mutex>
std::mutex m;
if (m.try_lock()) {  // ключ наш
    balance += 100;
    m.unlock();
} else {
    // занято — сделаем другое
}
```

Осторожно — ошибки

- Отдаёт ключ только тот поток, который его взял.
- Второй lock() того же мьютекса в том же потоке — UB. Обычно поток вечно ждёт сам себя.
- Мьютекс нельзя копировать — замок у всех один.

Забытый unlock() вешает всех → lock_guard

ОШИБКА: ВРУЧНУЮ — ОПАСНО

```
void Withdraw(long long x) {
    m_mutex.lock();
    if (x > m_balance)
        return;   // ушли без unlock!
    m_balance -= x;
    m_mutex.unlock();
}
```

Ранний return или исключение — до unlock() не дошли. Ключ занят навсегда, остальные потоки ждут вечно.

ВЕРНО: С LOCK_GUARD

```
void Withdraw(long long x) {
    std::lock_guard lock(m_mutex);
    if (x > m_balance)
        return;   // ключ отдаст сам
    m_balance -= x;
}  // здесь деструктор: unlock
```

Конструктор взял ключ, деструктор отдал — при любом выходе: return, исключение, конец функции.

RAII — объект сам отдаёт ресурс в деструкторе. Тот же приём, что std::jthread в лекции 2.
Правило курса: **lock() / unlock() руками не пишем.**

Мьютекс защищает данные, а не строчки

```
class Account {
public:
    void Deposit(long long x) {
        std::lock_guard lock(m_mutex);
        m_balance += x;
    }
    long long GetBalance() const {
        std::lock_guard lock(m_mutex);
        return m_balance;  // копия
    }
private:
    mutable std::mutex m_mutex; // охраняет m_balance
    long long m_balance = 0;
};
```

1. Все обращения — под замком

И запись, и чтение — под одним и тем же мьютексом.

2. Мьютекс — рядом с данными

Член того же класса, private. Снаружи до данных не добраться без замка.

3. mutable — для const-методов

const-метод не может менять поля, а lock() меняет мьютекс. mutable это разрешает.

4. Наружу — только копию

Ошибка — вернуть ссылку или указатель на m_balance: по ним прочтут без замка.

Одно правило — одна критическая секция

ПРАВИЛО БАНКА

наличные + счета = 1000 всегда

до Deposit(100)

`600 + 400 = 1000`

→

посередине (на миг)

`500 + 400 = 900`

другой поток: «пропало 100!»

→

после

`500 + 500 = 1000`

ОШИБКА: ДВА ОТДЕЛЬНЫХ ЗАМКА

```
void Deposit(long long x) {
    {   std::lock_guard a(m_cashMutex);
        m_cash -= x; }        // 600 → 500
    // ← замки отпущены: видно 900
    {   std::lock_guard b(m_accMutex);
        m_balance += x; }     // 400 → 500
}
```

ВЕРНО: ОДИН ЗАМОК НА ОБА ИЗМЕНЕНИЯ

```
void Deposit(long long x) {
    std::lock_guard lock(m_mutex);
    m_cash -= x;        // 600 → 500
    m_balance += x;     // 400 → 500
}   // замок отпущен только здесь
long long GetTotal() const {
    std::lock_guard lock(m_mutex); // тот же
    return m_cash + m_balance; // 1000
}
```

Если правило связывает несколько переменных — меняйте их **под одним замком**.

Безопасные методы ≠ безопасный интерфейс

Потокобезопасный — можно звать из разных потоков сразу. **Интерфейс** — методы класса для пользователя.

ОШИБКА: ПРОВЕРКА И СНЯТИЕ — ДВА ВЫЗОВА

```
// на счёте 100, два клиента
if (acc.GetBalance() ⩾ 80)  // замок
    // ← окно: замок свободен
    acc.Withdraw(80);       // замок
```

Каждый метод сам берёт и отдаёт замок. Между вызовами второй клиент тоже проходит проверку.

Итог: снова −60

ВЕРНО: ОДИН МЕТОД TRYWITHDRAW

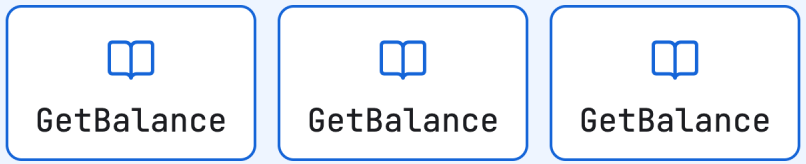
```
bool TryWithdraw(long long amount) {
    std::lock_guard lock(m_mutex);
    if (m_balance < amount)
        return false;   // не хватает
    m_balance -= amount;
    return true;        // сняли
}
```

Проверка и снятие — под одним замком. Второй клиент получит false, баланс останется 20.

«Проверил» и «сделал» — **в одном методе**. Иногда ради этого меняют интерфейс класса.

shared_mutex: читают многие, меняет один

Читают — сразу многие



GetBalance берёт **std::shared_lock** — замок для чтения. Он ничего не меняет, поэтому таких пускают вместе.

Меняет — только один



Deposit берёт **std::unique_lock** — замок для одного, как lock_guard. Входит, когда внутри пусто; остальные ждут.

```
#include <mutex>          // unique_lock
#include <shared_mutex>   // shared_lock
mutable std::shared_mutex m_mutex;
// GetBalance() const — только читает:
std::shared_lock lock(m_mutex);
// Deposit, Withdraw — меняют:
std::unique_lock lock(m_mutex);
```

Когда это быстрее mutex?

Только если чтений **намного больше**, чем записей, **и** каждое чтение **долгое**.

Для короткого GetBalance обычный mutex, как правило, не хуже: shared_mutex сам по себе дороже.

Сначала измерьте, потом меняйте.

Цена мьютекса: потоки стоят в очереди



Пока T1 в секции, T2, T3 и T4 просто ждут.

Секция — последовательная часть программы. Закон Амдала (лекция 1): чем она длиннее, тем меньше ускорение.

ОШИБКА: ДОЛГОЕ — ПОД ЗАМКОМ

```
// Deposit(x) с комиссией:
std::lock_guard lock(m_mutex);
long long fee = CalcFee(x);  // долго
m_balance += x - fee;
std::cout << "+" << x << '\n';  // долго
// …а T2, T3, T4 всё это время ждут
```

ВЕРНО: ПОД ЗАМКОМ ТОЛЬКО ДАННЫЕ

```
long long fee = CalcFee(x);  // без замка
{
    std::lock_guard lock(m_mutex);
    m_balance += x - fee;  // только это
}   // здесь замок отпущен
std::cout << "+" << x << '\n';  // без замка
```

Секция — как можно короче: под замком только изменение данных.

Ничего долгого под замком: вывод на экран, файлы, сеть, sleep. Замер: 4 потока с замком на каждое +1 — в ~70 раз медленнее одного потока без замка.

std::atomic: одна операция — неделимая

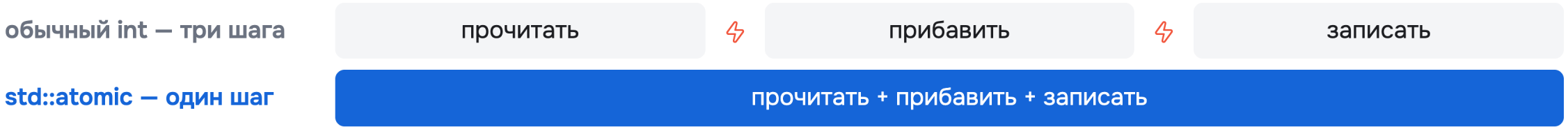
```
#include <atomic>
// в <> — тип значения: одно число long long
std::atomic<long long> balance{0};
// прочитать+прибавить+записать — одним целым:
balance.fetch_add(100);
balance += 100;                  // то же самое
long long b = balance.load();    // прочитать
balance.store(0);                // записать
```

Атомарная = неделимая

Другой поток видит либо «до», либо «после». Середины нет.

Верно: гонки данных нет

Если все обращения к balance атомарные, гонки данных нет — даже без мьютекса. И обычно так быстрее.



Предел atomic: две операции — снова гонка

```
std::atomic<long long> balance{100};
// два потока одновременно:
if (balance.load() ⩾ 80)    // атомарно
    balance.fetch_sub(80);   // атомарно
// ↑ а между ними — окно!
```

Ошибка: баланс −60

Оба потока проверили 100 ≥ 80 — да. Оба сняли по 80: 100 → 20 → −60.

Каждая операция неделима, а пара из двух — нет. Та же ошибка «проверил, потом сделал».

	std::atomic	мьютекс
Что защищает	одну переменную	сколько угодно данных
Сколько шагов	одну операцию	сколько угодно шагов
Примеры	счётчик операций, флаг «готово»	баланс + наличные, «проверил и снял»
В лабе 3	счётчик GetOperationsCount	всё остальное